THE UNIVERSITY OF WESTERN AUSTRALIA

Web-servers and Server-Side Rendering

CITS3403 and CITS5505 - Agile Web Development

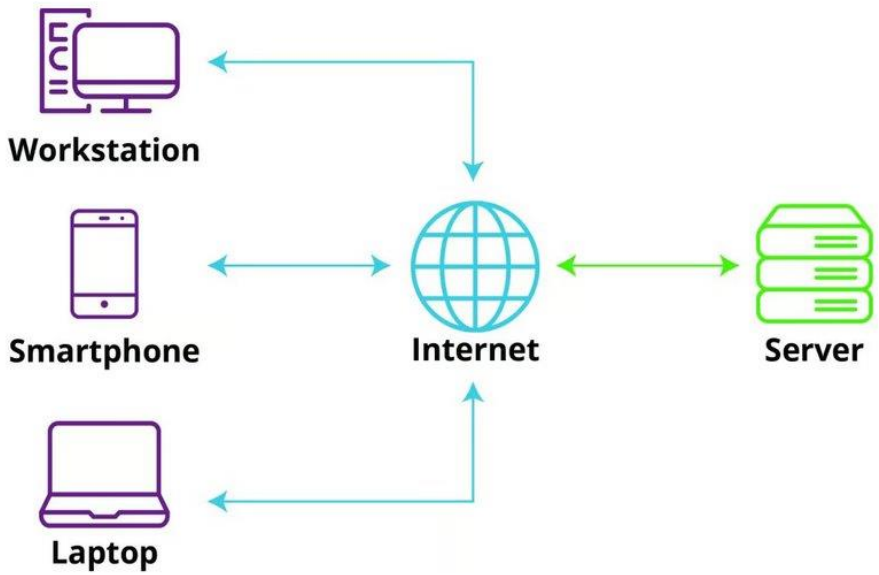
Adapted from the Flask Mega-Tutorial, by Miguel Grinberg:
https://blog.miguelgrinberg.com/post/the-flask-mega-tutorial

Semester 1, 2026

Full-stack development

What is a web server?

- Up until now, we have only looked at programs running in the browser, but we will now shift focus to programs running on the server.



- A web server is simply a program running on a computer connected to the internet that responds to requests from other computers on the network.

What does a web server do?

- The web-server program follows the event-driven programming paradigm:
    1. It waits until it receives a HTTP request from a client.
    2. Based on the HTTP request it performs some logic to generate or store data.
    3. It constructs a reply to the HTTP request.
    4. It sends the reply to the client and goes back to waiting.

- Backend frameworks abstract away steps 1 and 4 and instead allow you to focus on the application logic in steps 2 and 3.

- They also provide support for:
    1. generating HTML pages dynamically based on data held on the server
    2. interacting with databases to store that data.
    3. etc. etc.

- There are many different backend frameworks, but they all fulfil a similar role.

Different backend stacks

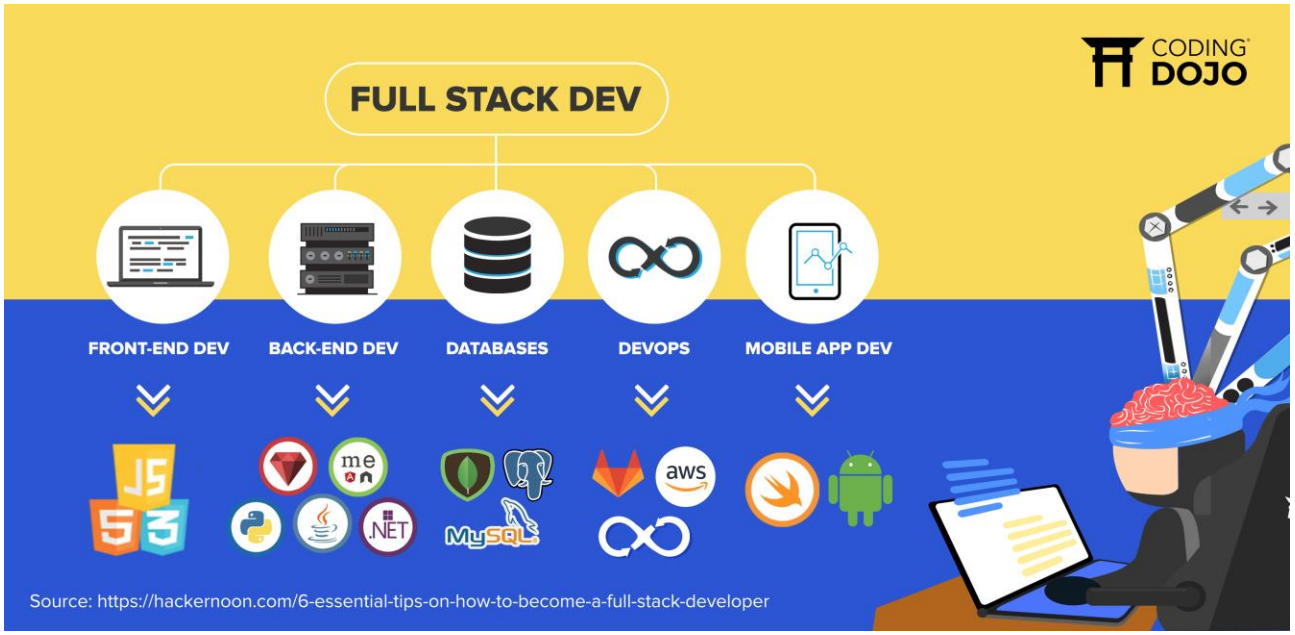
- The "backend stack" consists of the server operating system, the backend framework, the choice of database, the programming language etc.

- Examples of backend stacks in use:
  - LAMP (Linux, Apache, MySQL and PHP)
  - Ruby on Rails (Linux, Rails, <database> and Ruby)
  - Django (Linux, Django, <database> and Python)

- We are going to use the following backend stack in this unit:
  - Linux or Windows for the server OS.
  - Python for the programming language for writing the server logic.
  - Flask, a Python micro framework, for implementing the core web server loop and other functionality.
  - SQLite, a lightweight database management system.

Full-stack development

- Full-stack development involves knowing a full set of technologies used for both front-end and the back-end. Most developers are specialised in one part of the stack.



- The stack refers to a complete list of technologies needed to implement both a front-end and a backend.

Flask basics

Getting started with Flask

- Flask is a micro-framework that can be used to create a server program that will run on any machine and has few dependencies.

- You will need python3 and pip installed in your operating environment.

- Initialise a new Python virtual environment and activate it.

```
UNIWA\00112652@dep71351vm:~/Code/AgileWebDev2025$ python3 -m venv application-env
UNIWA\00112652@dep71351vm:~/Code/AgileWebDev2025$ source application-env/bin/activate
```

- Now install Flask. Any required modules will be kept in the virtual-environment.

```
(application-env) UNIWA\00112652@dep71351vm:~/Code/AgileWebDev2025$ pip install flask
Collecting flask
```

- You can now run flask by typing `flask run`, but the app doesn't know what to run.

```
Error: Could not locate a Flask application.
```

Creating the Flask server

- To create a minimal Flask server application, we create a file `__init__.py` in a folder called `app` as follows:

```
app > __init__.py > ...
from flask import Flask

app = Flask(__name__)

if __name__ == '__main__':
    app.run()
```

- In this file, we have created a new instance of Python class `Flask` and the resulting object implements the basic server loop of listening for HTTP requests.
- If we now rerun the `flask run` on the command line, we now get the following output:

```
 * Debug mode: off
WARNING: This is a development server.
 * Running on http://127.0.0.1:5000
```

Accessing the server

- The key line is this:

```
* Running on http://127.0.0.1:5000
```

- This is a URL that says the server is running at IP address 127.0.0.1 on port 5000.

- The IP address 127.0.0.1 is a special IP address that always refers to your own computer and is colloquially referred to as localhost.

- You can therefore access the Flask server in your browser by going to either `http://127.0.0.1:5000` or `http://localhost:5000/`.

- But we have not yet setup the server to provide any content.

Not Found

The requested URL was not found on the server.

Serving static files

- Flask servers usually store static files to serve to clients in the `app/static/` folder, e.g.

app > static > test.html > ...

```html
<!DOCTYPE html>
<html>
<head>
    <title>Hello, World!</title>
</head>
<body>
    <h1>Hello, World!</h1>
</body>
</html>
```

localhost:5000/static/test.html

Hello, World!

- Now if we restart the server and go to `http://localhost:5000/test.html` we get the above.

- Although it seems similar, to the "Hello world!" example we had at the start of the course when we were first learning HTML, **the actual implementation is totally different**!

- Instead of the browser opening a local HTML file, the browser is now making a request to a server program (which happens to be on the same computer) and receiving the result.

Basic Flask application structure

- To create URLs that correspond to *functions* on the server we need to create Flask routes.

app > __init__.py > ...

```python
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello, World!'

if __name__ == '__main__':
    app.run()
```

- The file has a method to return ‘Hello world!’ that is *decorated* with `@app.route('/')`.

- The variable `app` is an instance of the class `Flask`. When it runs it listens for requests, and if the route matches a function's decorator, it executes that function.

- The return of the function automatically becomes the body of the HTTP response.

A more scalable application structure

- But this single file structure doesn't scale well.
- A better structure is to create a Python package that will contain all the code we need for the web app.
- The __init__.py file creates an instance of the Flask class and routes.py contains the request handlers.
- Finally, we usually have a file at the top level to import `app`.
- Now the `app` package can contain files for handling routes, modules, templates, tests and anything else required.

app/__init__.py: Flask application instance

```
from flask import Flask

app = Flask(__name__)

from app import routes
```

app/routes.py: Home page route

```
from app import app

@app.route('/')
def index():
    return "Hello, World!"
```

microblog.py: Main application module

```
from app import app
```

Accessing HTTP request data

- Flask automatically parses all the data associated with the HTTP request.
- You can access it manually via the special `request` context variable.
- Of interest is:
  - request.args - key/value pairs in the URL query string
  - request.form - key/value pairs from a HTML form (POST request)
  - request.method - the HTTP method used

app > roots.py > ...

```python
from flask import request
from app import app

@app.route('/user/')
def show_user_profile():
    username = request.args.get('username')
    return f'User %s' % username
```

Flask endpoints

- The functions in `routes.py` annotated with @app.route(…) are known as endpoints.
- An endpoint can be the destination of multiple URLs:

```
@app.route('/')
@app.route('/index')
def hello(userid):
    return 'Hello world!'
```

- They can also accept parameters using the `<variable>` syntax which then get automatically passed to the Python function, instead of having to manually retrieve them from `request`:

```
@app.route('/user/<username>')
def show_user_profile(username):
    # show the user profile for that user
    return 'User %s' % username
```

- Non-string parameters can be parsed automatically by using a converter:

```
@app.route('/user/<int:userid>')
def show_user_profile(userid):
    # show the user profile for that user
    return 'User %d' % userid
```

Endpoints, HTTP methods and url_for

- By default, endpoints are only available as GET requests.

- However, you can set them as end-points of various types using the `methods` parameter:

```
@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        do_the_login()
    else:
        show_the_login_form()
```

- The endpoints form the backbone of your web application, and you will frequently want to link to them in HTML. However, you don't want to have to duplicate the long URLs everywhere in your program.

- Therefore, you can use the function `url_for` to retrieve the URL for a given endpoint:

Getting endpoint URLs with url_for

- The endpoints form the backbone of your application, and you will use them frequently! However, you don't want to be writing the URLs everywhere in your application:
  - The URLs can be very long.
  - If the files in your directories change, the URL must change.
  - If you change a URL, you want to change it one place only.

- Therefore, you can use the function `url_for` to retrieve the URL given the name of an endpoint:

```
@app.route('/looping', methods=['GET', 'POST'])
def loop():
    thisURL = url_for('loop')
    return "<a href={}> Let's go back to {} </a>".format(thisURL, thisURL)
```

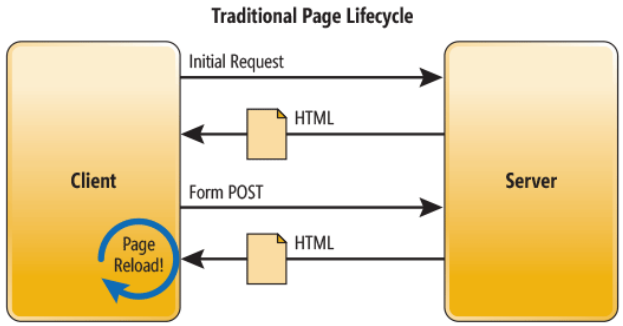
⇩

Let's go back to /looping

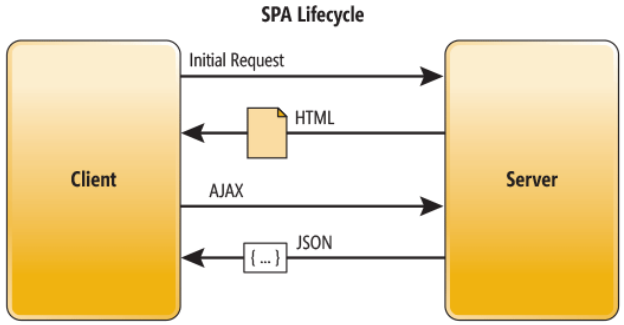
- Next question: how do we return complex web-pages from the server?

Server-side vs Client-side rendering

- There are two approaches to serving dynamic HTML:
  1. Server-side rendering - the server builds the HTML when it receives the request and sends it to the client.



  2. Client-side rendering - the server sends JavaScript and an HTML skeleton to the client, and the client can then request JSON and build the HTML using AJAX and jQuery.



- Server-side rendering is the traditional approach. However, client-side rendering is more flexible and allows greater support for non-browser devices. Flask supports both!

Flask server-side rendering

Basic server-side rendering

- The most basic approach to server-side rendering listens for requests and uses Python functions to directly build HTML pages to return as a response.

app/routes.py: Return complete HTML page from view function

```
from app import app

@app.route('/')
@app.route('/index')
def index():
    user = {'username': 'Miguel'}
    return '''
<html>
    <head>
        <title>Home Page - Microblog</title>
    </head>
    <body>
        <h1>Hello, ''' + user['username'] + '''!</h1>
    </body>
</html>'''
```

- However, this mixes the logic and the presentation!

Server-side templates

- A better approach is to use HTML templates that references variables, and a rendering function that will take a template and an assignment to those variables and builds the HTML dynamically.

```
33      {% for p in projects%}
34        <tr>
35          <td>{{p['team']}}</td>
36          <td>{{p['description']}}</td>
37          <td>{{p['lab']}}</td>
38          <td>{{p['time']}}</td>
39          {% if not current_user.is_anonymous %}
40            <td>
41            {% if p['project_id']== current_user.project_id %}
42            <a href='{{url_for("delete_project") }}'>delete</a>
43              <a href='{{ url_for("edit_project") }}'>edit</a>
44            {% endif %}
45            </td>
46          {% endif %}
47      </tr>
48      {% endfor %}
49    </table>
```

- Flask uses Jinja for this task, but there are many alternatives (pug, handlebars)

Getting started with Jinja

- We separate presentation and logic by having a template directory that contains HTML files annotated with placeholder variables distinguished by `{{ … }}` braces.

app/templates/index.html: Main page template

```
<html>
    <head>
        <title>{{ title }} - Microblog</title>
    </head>
    <body>
        <h1>Hello, {{ user.username }}!</h1>
    </body>
</html>
```

app/routes.py: Use render_template() function

```
from flask import render_template
from app import app

@app.route('/')
@app.route('/index')
def index():
    user = {'username': 'Miguel'}
    return render_template('index.html', title='Home', user=user)
```

- When a request is received Flask will look for the matching template (in the directory templates) and convert the template to pure HTML using `render_template` function.

- The values of the named arguments to this function are then substituted by Jinja into the locations indicated by the matching placeholder variables.

Dynamically Jinja templates

- Depending on the parameters passed, we may want to display the data differently.
- Jinja provides loops and conditionals to allow the display to adapt to the data.
- For example, it is common to pass in an array of objects and then present them in a table.
- Or we may want the display to vary depending on who is logged in.

```
33      {% for p in projects%}
34        <tr>
35          <td>{{p['team']}}</td>
36          <td>{{p['description']}}</td>
37          <td>{{p['lab']}}</td>
38          <td>{{p['time']}}</td>
39          {% if not current_user.is_anonymous %}
40            <td>
41            {% if p['project_id']== current_user.project_id %}
42            <a href='{{url_for("delete_project") }}'>delete</a>
43              <a href='{{ url_for("edit_project") }}'>edit</a>
44            {% endif %}
45            </td>
46          {% endif %}
47      </tr>
48      {% endfor %}
49    </table>
```

```
 9 @app.route('/')
10 @app.route('/index')
11 def index():
12   print('index')
13   if current_user.is_authenticated:
14     projects = get_all_projects()
15   else:
16     projects = []
17   return render_template('index.html', projects=projects)
```

Jinja conditional control statements

- The syntax for control statements is `{% ... %}`.

app/templates/index.html: Conditional statement in template

```
<html>
    <head>
        {% if title %}
        <title>{{ title }} - Microblog</title>
        {% else %}
        <title>Welcome to Microblog!</title>
        {% endif %}
    </head>
    <body>
        <h1>Hello, {{ user.username }}!</h1>
    </body>
</html>
```

- Conditionals use `if`, `else`, `elif`, as well as `endif`, since whitespace scoping doesn't work for HTML.

Jinja loop control statements 1

- We can also use `for` and `while` loops for iterating through collections.

```
@app.route('/')
@app.route('/index')
def index():
    user = {'username': 'Miguel'}
    posts = [
        {
            'author': {'username': 'John'},
            'body': 'Beautiful day in Portland!'
        },
        {
            'author': {'username': 'Susan'},
            'body': 'The Avengers movie was so cool!'
        }
    ]
    return render_template('index.html', title='Home', user=user, posts=posts)
```

Jinja loop control statements 2

- We can also use `for` and `while` loops for iterating through collections.

app/templates/index.html: for-loop in template

```
<html>
    <head>
        {% if title %}
        <title>{{ title }} - Microblog</title>
        {% else %}
        <title>Welcome to Microblog</title>
        {% endif %}
    </head>
    <body>
        <h1>Hi, {{ user.username }}!</h1>
        {% for post in posts %}
        <div><p>{{ post.author.username }} says: <b>{{ post.body }}</b></p></div>
        {% endfor %}
    </body>
</html>
```

Jinja inheritance

- Since we often want the titles, menus, footers in an application to be the same, we can have the templates inherit from each other.

app/templates/base.html: Base template with navigation bar

```
<html>
    <head>
      {% if title %}
      <title>{{ title }} - Microblog</title>
      {% else %}
      <title>Welcome to Microblog</title>
      {% endif %}
    </head>
    <body>
        <div>Microblog: <a href="/index">Home</a></div>
        <hr>
        {% block content %}{% endblock %}
    </body>
</html>
```

- This is done using the `{% block <blockName> %}` construct which indicates that the named block is left unspecified for other templates to fill in.

Jinja inheritance

- You can indicate that one template inherits from another by using the following syntax:

  {% extends <parentTemplateName> %}

- The contents of each named block can then be provided using the same block syntax:

app/templates/index.html: Inherit from base template

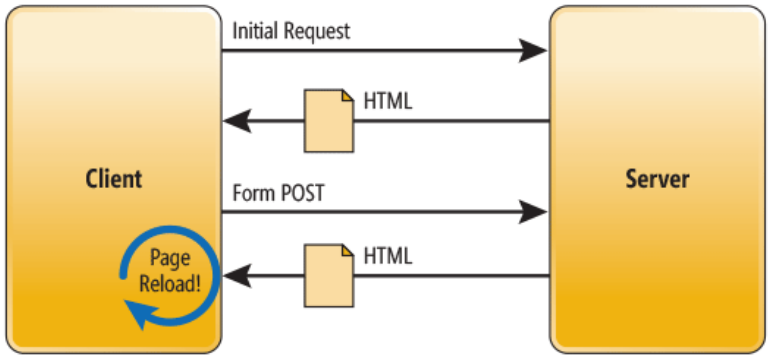
```
{% extends "base.html" %}

{% block content %}
    <h1>Hi, {{ user.username }}!</h1>
    {% for post in posts %}
    <div><p>{{ post.author.username }} says: <b>{{ post.body }}</b></p></div>
    {% endfor %}
{% endblock %}
```

- This is an instance of the general DRY principle: *don't repeat yourself*

Flask forms

Forms on the server

- Forms are used to move data from the client to the server, and when you hit the "Submit" button builds a POST request and sends it to the server.



- The server then (hopefully!) does something with the data and then returns a response. Often this involves directing the browser to navigate to a new page.

- We could access the form data in the HTTP request directly but is very manual...

- Flask uses the WTForms module to both insert the form into the Jinja template and then to validate POST requests from the rendered form. Install `flask-wtf` with `pip`.

Securing the form

- Web forms are vulnerable to attacks known as cross site request forgery (CSRF).
- The exact mechanism of this attack will be covered in the "Security" lecture.
- For now, it is enough to know that a server should include a secret key in every form it generates.

app > __init__.py > ...

```python
from flask import Flask

app = Flask(__name__)
app.config['SECRET_KEY'] = 'you-will-never-guess'
```

- For now, we will set this in the __init__.py, file
- This is <u>not secure</u> as this file will be pushed to GitHub!

- Again, we will cover better ways to store your secret keys in the "Security" lecture.

Data model for a form

- There are three parts to creating a form on the Flask server:
  1. Use WTForms to create a class describing the data model for the fields
  2. Update the Jinja template to describe how the form should render.
  3. Update the routes.py file with a function to handle the submitted form.

- Python classes for forms typically live in a new file `forms.py` in app:

app > forms.py > ...

```python
from flask_wtf import FlaskForm
from wtforms import IntegerField, BooleanField, SubmitField
from wtforms.validators import DataRequired

class LoginForm(FlaskForm):
    student_id = IntegerField('Student ID', validators=[DataRequired()])
    pin = IntegerField('PIN', validators=[DataRequired()])
    remember_me = BooleanField('Remember Me')
    submit = SubmitField('Sign in')
```

Rendering a form

- Jinja works with flask-wtf to put the appropriate input elements in the page.
- The `form.hidden_tag()` entry is used to protect against CSRF attacks
- The form elements are those defined by the `forms.py` class
- Attributes can be appended to the elements in brackets.
- The `url_for()` is used to reference the URL for the correct endpoint.

```
1  {% extends "base.html" %}
2
3  {% block content %}
4  <h2>Login</h2>
5
6    <form name='login' action='' method='post'>
7      <div class='form-group'>
8        {{form.hidden_tag()}}
9        <p>
10       {{ form.student_number.label }}<br>
11       {{ form.student_number(size=8) }}
12       {% for error in form.student_number.errors %}
13       <span style="color:red;">[{{ error}}]</span>
14       {% endfor %}
15       </p>
16       <p>
17       {{ form.pin.label }}<br>
18       {{ form.pin(size=4) }}
19       {% for error in form.pin.errors %}
20       <span style="color:red;">[{{ error}}]</span>
21       {% endfor %}
22       </p>
23       <p> {{form.remember_me() }} {{form.remember_me.label }}</p>
24       <p> {{ form.submit() }}</p>
25     </div>
26   </form>
27   <p>To register <a href={{ url_for('register') }}>click here</a></p>
28   {% endblock %}
```

Rendering a form

- The final generated HTML when the template is instantiated is:

```
<h2>Login</h2>

    <form name='login' action='', method='post'>
        <div class='form-group'>
            <input id="crsf_token" name="crsf_token" type="hidden" value="ImU2NzU50DlhMDg2YWE3NzE4ZWN">
            <p>
                <label for="student_number">Student Number</label><br>
                <input id="student_number" name="student_number" required size="8" type="text" value="">
            </p>
            <p>
                <label for="pin">Pin Code</label><br>
                <input id="pin" name="pin" required size="4" type="password" value="">
            </p>
            <p>
                <input id="remember_me" name="remember_me" type="checkbox" value="y">
                <label for="remember_me">Remember Me</label>
            </p>
            <p>
                <input id="submit" name="submit" type="submit" value="Sign In">
            </p>
        </div>
    </form>
```

Processing a form

- To process a form, we configure a route for the POST method.

```
from flask import render_template, flash, redirect

@app.route('/login', methods=['GET', 'POST'])
def login():
    form = LoginForm()
    if form.validate_on_submit():
        flash('Login requested for user {}, remember_me={}'.format(
            form.username.data, form.remember_me.data))
        return redirect('/index')
    return render_template('login.html', title='Sign In', form=form)
```

- We create an instance of the form class, for both rendering and wrapping posted data.
- A GET request won't validate, so it will jump to the last line, and render the page.
- If a POST request validates, a flash message is created, and the page is redirected to the index.
- This does not actually check a user's password; to do so we need a database (see "Databases" lecture).

Displaying flash messages

- If a form doesn't validate, the errors are accessible in a list but are rendered server side. Faster client-side validation can be applied using JavaScript.

- The flash messages are just a list that can be accessed by other pages.

```
6   <body>
7       <div>
8           Microblog:
9           <a href="/index">Home</a>
10          <a href="/login">Login</a>
11      </div>
12
13      {% with messages = get_flashed_messages() %}
14          {% if messages %}
15              <ul>
16                  {% for message in messages %}
17                      <li>{{ message }}</li>
18                  {% endfor %}
19              </ul>
20          {% endif %}
21      {% endwith %}
```

Other

Context variables

- Flask uses deep Python magic to supply special variables known as context variables to various methods, without passing them explicitly as a parameters to functions:

Variable name	Available	Purpose
g	Always	Stands for "global". An object that contains the global context of the application.
session	Inside a @app.route function	Uses (secure) cookies to transfer data between multiple requests from the same user.
request	Inside a @app.route function	Contains the data from the current HTTP request.

Debugging in the Flask shell

- The Flask shell is a useful way to test small functions and their integration with flask, without using a browser.

```
from app import app, db
from app.models import Student, Project, Lab

@app.shell_context_processor
def make_shell_context():
  return {'db':db, 'Student':Student, "Project":Project, 'Lab':Lab}
```

```
(virtual-environment) drtnf@drtnf-ThinkPad:$ export SECRET_KEY='poor_secret'
(virtual-environment) drtnf@drtnf-ThinkPad:$ echo $SECRET_KEY
poor_secret
(virtual-environment) drtnf@drtnf-ThinkPad:$ flask shell
Python 3.6.7 (default, Oct 22 2018, 11:32:17)
[GCC 8.2.0] on linux
App: app [production]
Instance: /Dropbox/ArePricks/Dropbox/Tim/teaching/2019/CITS3403/pair-up/instance
>>> print(app.config['SECRET_KEY'])
poor_secret
>>>
```

- It loads the flask app, and all the dependencies, but doesn't need the server running. You can set the shell context to have variables predefined when you start the shell.

Debug mode

- Debug mode is also useful. Set the system variable FLASK_DEBUG=1 to get a trace of the errors when the server crashes.

```
(venv) $ export FLASK_DEBUG=1
```

builtins.NameError

```
NameError: name 'FlaskForm' is not defined
```

Traceback (most recent call last)

```
File "/Dropbox/ArePricks/Dropbox/Tim/teaching/2019/CITS3403/pair-up/virtual-environment/lib/python3.6/site-packages/flask/_compat.py", line 35, in reraise
  raise value
File "/Dropbox/ArePricks/Dropbox/Tim/teaching/2019/CITS3403/pair-up/pair-up.py", line 1, in <module>
  from app import app, db
File "/Dropbox/ArePricks/Dropbox/Tim/teaching/2019/CITS3403/pair-up/app/__init__.py", line 14, in <module>
  from app import routes,models
File "/Dropbox/ArePricks/Dropbox/Tim/teaching/2019/CITS3403/pair-up/app/routes.py", line 4, in <module>
  from app.forms import LoginForm, RegistrationForm, ProjectForm
File "/Dropbox/ArePricks/Dropbox/Tim/teaching/2019/CITS3403/pair-up/app/forms.py", line 7, in <module>
  class LoginForm(FlaskForm):
```

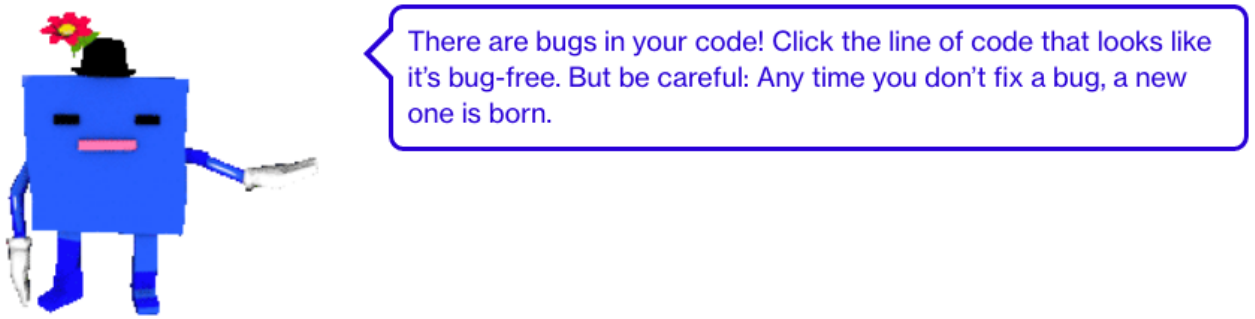
NameError: name 'FlaskForm' is not defined

Suggested reading

Read “What is Code” by Paul Ford:

http://www.bloomberg.com/graphics/2015-paul-ford-what-is-code/

This is a very good overview with software development in general, with lots of web-development specifics later on. Note that it was written before LLMs.




```
var salesPlusFour = 4 + sales;
```

```
var salesPlusFour = "4" + sales;
```

```
for (var i = 0; i < 10; i++)
```

```
for (var i = 0; i < 10 i++)
```